



Programovacie jazyky

Haskell vs. Racket, statická vs. dynamická kontrola, eval

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

20. mája 2018

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

eval, quasiquote, unquote, ...

Funkcie eq? a equal?

- `(eq? v1 v2)` vracia `#t` ak v_1 a v_2 odkazujú na ten istý objekt, inak vracia `#f`.
 - `(eq? "Ahoj" "Ahoj")` $\rightsquigarrow$ `#t`
 - `(eq? "ab" (string-append "a" "b"))` $\rightsquigarrow$ `#f`
 - `(eq? (cons 1 2) (cons 1 2))` $\rightsquigarrow$ `#f`
 - `(eq? 2 2.0)` $\rightsquigarrow$ `#f`
- `(equal? v1 v2)` pre typy ako reťazce, páry, meniteľné páry, a iné porovnáva podľa obsahu. Pre páry je rovnosť rekurzívna.
 - `(equal? "Ahoj" "Ahoj")` $\rightsquigarrow$ `#t`
 - `(equal? "ab" (string-append "a" "b"))` $\rightsquigarrow$ `#t`
 - `(equal? (cons 1 2) (cons 1 2))` $\rightsquigarrow$ `#t`
 - `(equal? 2 2.0)` $\rightsquigarrow$ `#f`
- `(= v1 v2)` vracia `#t` práve vtedy ak $v_1 = v_2$. Ak v_1 a v_2 nie sú čísla, vráti chybu: (expected: number?).
 - `(= 2 2.0)` $\rightsquigarrow$ `#t`

Symboly

- **Symbol** je atomická hodnota, ktorá sa vypisuje ako identifikátor so symbolom ' na začiatku.
- Výraz, ktorý začína symbolom ' a pokračuje identifikátorom vyprodukuje symbol.
 - 'a $\rightsquigarrow$ 'a
 - (symbol? 'a) $\rightsquigarrow$ #t
- (string->symbol s) konvertuje reťazec s na symbol 's'.
 - (eq? 'a (string->symbol "a")) $\rightsquigarrow$ #t
 - (symbol->string 'a) $\rightsquigarrow$ "a"
 - (eq? (symbol->string 'a) "a") $\rightsquigarrow$ #f
 - (equal? (symbol->string 'a) "a") $\rightsquigarrow$ #t
- symboly rozlišujú malé a veľké písmená
- porovnanie symbolov pomocou eq? a equal? je $O(1)$
 - n rozdiel od equal? na reťazcoch, čo je $O(n)$

Funkcia eval

Jazyky ako Racket, Scheme, LISP, JavaScript, atď. majú funkciu eval, ktorá umožňuje:

- vytvoriť za behu nové dáta
 - Racket – zahniezdený zoznam
 - `(eval '(print "Ahoj"))`
 - JavaScript – reťazec znakov
 - `eval("alert('Ahoj.')`
- a potom tieto dáta zmeniť na program a vykonať ho

Keďže nevieme, aké dáta bude chcieť program vytvoriť a následne zmeniť na program, potrebujeme, aby súčasťou behového (runtime) prostredia bola aj implementácia jazyka. Tá môže byť v podobe:

- interpretera (veľmi časté riešenie)
- kompilátora
- kombinácia oboch možností

- `(quote v)` je to isté ako `'v`
- `quote` zabráni interpretácii symbolov ako identifikátorov a zoznamov ako volaní funkcií
 - `(quote apple)` $\rightsquigarrow$ `'apple`
 - `(quote (1 2 . (3)))` $\rightsquigarrow$ `(1 2 3)`
 - `(quote 42)` $\rightsquigarrow$ `42`

quasiquoting

- `(quasiquote v)` je to isté ako `'v`
- `(unquote v)` je to isté ako `,v`
- **quasiquote** pracuje podobne ako **quote**, ale pokiaľ narazí na `(unquote v)`, tak vyhodnotí `v` a nahradí ním `(unquote v)`.
 - `'(1 2 ,(+ 1 2) ,(- 5 1))` $\rightsquigarrow$ `'(1 2 3 4)`
 - `'(1 2 ,(+ 1 2) ,(- 5 1))` $\rightsquigarrow$ `'(1 2 ,(+ 1 2) ,(- 5 1))`
- v iných jazykoch (napr. JavaScript) využívame **string interpolation**
 - `var n = 42;`
 - `"foo${n}bar"` $\rightsquigarrow$ `"foo42bar"`
 - `'foo${n}bar'` $\rightsquigarrow$ `"foo42bar"`

Haskell vs. Racket

Haskell z pohľadu Racket programátora

Haskell je akoby podmnožina Racketu. Existujú programy, ktoré Haskell neskompiluje ale Racket áno.

- veľa z nich obsahuje chyby, ktoré Racket zistí až za behu:

```
1 (define (f x) (+ x (car x)))
```

```
1 (define (g x) (+ x x)) ; ok  
2 (define (h x) (g (cons x 2)))
```

- v programoch, ktoré Haskell povoľuje, nepotrebujeme funkcie ako `number?`, ... (všetko má staticky určený typ)
- ale iné môžu byť užitočné:

```
1 (define (f x) (if (> x 0) "OK" #f))  
2 (define xs (list 1 #t "hi"))  
3 (define y (f (car xs)))
```

Racket z pohľadu Haskell programátora

- Racket akoby mal jeden veľký dátový typ **RT**. Všetky hodnoty sú práve tohto typu RT:

```
1 data RT = I Integer
2         | S String
3         | P (RT, RT)
4         | F ([RT] -> RT)
5         | ...
```

- Konštruktory Racket aplikuje implicitne. Každá hodnota je označovaná (tagged) svojím typom.
 - Keď v Rackete napíšeme 42, interne sa reprezentuje ako

Int 42 v Haskell:

IntTag	42
--------	----

- základné operácie implicitne kontrolujú značku (tag):

```
1 car x = case x of {P (a,_) -> a ; _ -> error "..."}
2 isPair x = case x of {P (_,_) -> True; _ -> False}
```

Statická a dynamická kontrola

Definícia

Statická kontrola je akákoľvek činnosť, ktorej úlohou je zamietnuť zlý program po tom, ako sa úspešne sparsoval (vytvoril sa AST), ale ešte pred tým, než sa spustí.

- súčasťou def. prog. jazyka je, aké statické kontroly vykonáva
 - môžu existovať nástroje pre dodatočnú statickú kontrolu
 - napríklad lint pre jazyk C
- typický prístup k stat. kontrole je zaviesť systém typov s cieľom:
 - nájsť chyby ako nesprávne typy argumentov `4/"2"`, `f v`,
a `f` nie je funkcia, `if` za ktorým nie je bool. výraz, ...
 - vynútenie abstrakcie
 - vyhnúť sa kontrole za behu
- aj dynamicky typované jazyky môžu robiť určité stat. kontroly
 - použitie nedefinovanej premennej

Haskell – nedostatky statickej kontroly

- v jazyku Haskell stat. kontrola nedeteguje nasledovné chyby:
 - zavolanie `head` na prázdnom zozname
 - delenie nulou
 - ...
 - vedeli by sme pomocou `dependent types`
- žiadna statická kontrola nemôže detegovať logické chyby:
 - preklep v názve premennej: `x` namiesto `y`
 - za predpokladu, že oba majú rovnaký typ a sú použité
 - nesprávna podmienka v `if`
 - nesprávny algoritmus

Statická kontrola – lekár pre program

Úlohou stat. kontroly je diagnostikovať, či má program chorobu X . Program má chorobu X , ak existuje vstup, na ktorom spraví X .

- stat. kontrola je **korektná** (sound) vzhľadom na X , ak deteguje každý program chorý na X a neprepustí ho
 - žiadne falošne negatívne výsledky testov
- stat. kontrola je **úplná** (complete) vzhľadom na X , ak každý program, ktorý netrpí X , prejde kontrolou bez nálezu X
 - žiadne falošne pozitívne výsledky testov
 - bez rank-2 typov Haskell nepustí: $f\ x = (x, 1)$;
 $g\ f\ x\ y = (f\ x, f\ y)$; $g\ f\ \mathbf{True}\ 'a'$

Poznámka

Moderné vlastnosti typových systémov (ako napr. rank-2 typy) majú za cieľ znížiť počet falošne pozitívnych výsledkov, t.j. zväčšiť úplnosť.

Poznámka

Statická kontrola pre skoro každé netriviálne X nemôže súčasne spĺňať nasledovné podmienky:

- *je korektná (sound) vzhľadom na X*
- *je úplná (complete) vzhľadom na X*
- *na každom programe skončí v konečnom čase*

Poznámka

Inak povedané, to, či program trpí X je nerozhodnuteľný problém skoro pre každé netriviálne X .

Kompromis – nekorektný systém

Keďže nemôžeme mať všetko a chceme aby kompilátor vždy skončil, tak musíme upustiť od korektnosti alebo úplnosti. Upustíme teda od korektnosti. Čo s tým môžeme spraviť?

- Vložíme dyn. kontrolu, ktorá bude za behu kontrolovať, či sa nejde udiť X . Ak áno, program zastavíme a vyhlásime chybu.
- Povolíme, aby sa stalo X , aj keď v niektorých prípadoch tomu vieme zabrániť. (Např. delenie nulou vráti ∞ .)
 - Najhoršie je, ak povolíme po nastatí X akékoľvek (nedefinované) správanie (např. C).

Slabá (weak) kontrola

Špecifikácia jazyka povoľuje, aby niektoré programy, ktoré prešli statickou kontrolou, mali po spustení nedefinované správanie.

- nedefinované správanie $\Rightarrow$ môže sa stať čokoľvek
 - napr. buffer-overflow
- Záchranou je dyn. kontrola, ale v praxi sa nie vždy robí.
 - Java kontroluje za behu indexovanie do poľa, ale C nie.

Prečo niektoré jazyky nerobia ani statickú, ani dynamickú kontrolu?

- zjednodušuje to implementáciu kompilátora/interpretera
- vynechanie dynamických kontrol zvyšuje rýchlosť
- kompilátor negeneruje do kódu informácie potrebné (napr. veľkosť poľa) pre dyn. kontrolu (šetrí sa priestor a výkon)

Ak by ľudia chybovali menej ako počítače, tak by to dávalo zmysel.

- Linux kernel 4.16.8: 65 693 súborov, 26 267 697 riadkov kódu.
- Podľa **CVE** kernel mal za rok 2017 približne 472 zraniteľností.

Kontrola v jazyku Racket

- Racket nemá slabú kontrolu ako C
 - väčšinu vecí kontroluje za behu (dynamická kontrola)
 - makrá a nedefinované premenné rieši staticky
 - Optimalizácia: niektoré dynamické kontroly môžu byť vynechané, ak statická analýza potvrdí, že ich netreba.
- dynamická kontrola môže byť praktická
 - cons-bunka môže obsahovať čokoľvek
 - Haskell dovoľuje len `a : [a]`
 - všetko okrem `#f` je pravdivé
 - zoznam môže
 - byť ľubovoľne zahniezdený
 - obsahovať rôzne typy
 - ...

Kontrola v jazyku JavaScript

- "ahoj" + "svet" $\rightsquigarrow$ "ahojsvet"
- "ahoj" + 1 $\rightsquigarrow$ "ahoj1"
- indexovať za rozsah poľa je dovolené:
 - a[10] $\rightsquigarrow$ **undefined**
 - a[10] = 7 $\rightsquigarrow$ rozšírenia poľa
 - pridané prvky (okrem a[10]) budú **undefined**
- **function** sum(a,b) {**return** a+b} môžeme zavolať s ľubovoľným počtom argumentov:
 - sum("X") $\rightsquigarrow$ "Xundefined"
 - sum(1,3,7) $\rightsquigarrow$ 4

Poznámka

Toto nie je o statickej vs. dynamickej kontrole, ale o sémantike jazyka.

Statická vs. dynamická kontrola

Dynamická kontrola je praktickejšia

Dynamická typová kontrola štandardne dovoľuje heterogénne zoznamy, funkcie vracajúce String alebo Integer,

- dynamická kontrola: môžeme implementovať priamo

```
1 (define (f x)
2   (if (> x 0) (+ x x) "hi"))
3
4 (let ([ans (f 10)])
5   (if (number? ans)
6       (number->string ans)
7       ans))
```

- statická kontrola: viac práce, musíme zaviesť vlastný typ

```
1 data T = I Int | S String
2 f x = if x > 0 then I (x + x)
3       else S "hi"
4 y = case f 10 of
5   I i -> show i
6   S s -> s
```

Statická kontrola je praktickejšia

Pri statickej kontrole môžeme predpokladať, že argumenty majú správny typ.

```
1 cube x = x * x * x
2 cube 7
```

Pri dynamickej kontrole musíme ručne kontrolovať typ argumentu na začiatku funkcie (inak sa chyba môže prejaviť oveľa neskôr):

```
1 (define (cube x)
2   (if (not (number? x))
3       (error "not a number")
4       (* x x x)))
5
6 (cube 7)
```

Racket má za týmto účelom systém kontraktov (nespomínali sme).

Statická kontrola odmieta rozumné programy

Každá zložitejšia korektná statická kontrola nie je úplná, t.j. odmieta niektoré rozumné programy.

```
1 (define (f x) (cons x 1))
2
3 (define (g f x y)
4   (cons (f x) (f y)))
5
6 (g f #t "a")
```

Analogický kód v jazyku Haskell neprejde typovou kontrolu (bez rank-2 typov):

```
1 f x = (x, 1)
2 g f x y = (f x, f y)
3 g f True "a"
```

Statická kontrola umožňuje voliteľné značkovanie

Automatické značkovanie (tagging) všetkých hodnôt pri dynamickej kontrole stojí čas, priestor a prináša neskorú detekciu chýb.

Ak je to nutné, tak pri stat. kontrole si môžeme vytvoriť vlastný variantný typ a vo vybraných prípadoch explicitne značkovať.

```
1 data T = I Int | S String
2 f x = if x > 0 then I (x + x)
3           else S "hi"
4 y = case f 10 of
5     I i -> show i
6     S s -> s
```

V extrémnom prípade zavedieme všeobjímajúci typ **RT**.

Statická kontrola zachytí chyby skôr

Statická kontrola nájde veľa jednoduchých chýb počas kompilácie.

- keďže tieto chyby budú vždy odhalené, netreba ich testovať
- môžeme dávať menší pozor a spoliehať sa na statickú kontrolu
- dynamická kontrola nájde chybu až za behu

```
1 (define (pow x) ; curried
2   (lambda (y)
3     (if (= y 0)
4         1
5         (* x (pow x (sub1 y)))))
6 ; nie (pow x y) ale ((pow x) y)
```

- statická kontrola už počas kompilácie
(vďaka statickému typovému systému)

```
1 pow x y = if y == 1
2   then 1 -- malo byť pow x (y - 1)
3   else x * pow (x, y - 1)
```

Statická kontrola nájde iba jednoduché chyby

Aj pri statickej kontrole stále musíme písať testy. Tieto testy však okrem logických chýb objavia aj jednoduché chyby.

- dynamická kontrola nenájde chybu (+ namiesto ×)

```
1 (define (pow x)
2   (lambda (y)
3     (if (= y 0)
4         1
5         (+ x ((pow x) (sub1 y))))))
```

- statická kontrola, tiež nenájde chybu (+ namiesto ×)

```
1 pow x y =
2   if y == 1
3   then 1
4   else x + pow x (y - 1)
```

Statická kontrola generuje rychlejší kód

- podpora za behu (runtime):
 - nemusí ukladať značky (tags)
 - nemusí kontrolovať značky
- vlastný zdrojový kód:
 - nemusí ručne kontrolovať typy argumentov
 - po volaní nemusí kontrolovať typ návratovej hodnoty

Tým sa šetrí čas aj priestor počas behu.

Dynamická kontrola nie je až taká pomalá

- implementácia jazyka:
 - môže využívať rôzne optimalizácie k odstráneniu zbytočných dynamických kontrol. Napríklad:
- vlastný zdrojový kód:
 - nemusí ručne obchádzať obmedzenia typového systému

```
1 (let ([x (+ y y)]) (* x 4))
```

- Premennú *y* stačí kontrolovať iba raz (či je číslo).
- Premennú *x* netreba pri násobení kontrolovať, lebo je výsledkom sčítania, ktoré vždy vracia číslo.

```
1 (define (f x) (cons x 1))  
2  
3 (define (g f x y) (cons (f x) (f y)))  
4  
5 (g f #t "a")
```

Dynamická kontrola uľahčuje znovupoužiteľosť kódu

Bez obmedzujúceho statického typového systému sa dá viac kódu znovupoužiť s iným typom dát.

- ak používame cons-bunky pre všetko, knižnice pracujúce s cons-bunkami sú veľmi znovupoužiteľné
- knižnice kolekcií v statických jazykoch sú tiež veľmi znovupoužiteľné, ale často vyžadujú komplikované typy
- ...

Statická kontrola uľahčuje znovupoužiteľosť kódu

- moderné statické typové systémy umožňujú rozumnú mieru znovupoužita
 - generics, ad-hoc polymorfizmus, hierarchie tried, ...
- používanie cons-buniek pre všetko vedie k ťažko odladiteľným chybám (ak sa pomýlime)
 - statická typová kontrola umožňuje:
 - použiť rôzne typy pre rôzne koncepty
 - zabrániť nesprávnemu použitiu knižníc

Dynamická kontrola je lepšia pre prototypovanie

Spočiatku, pri prototypovaní, obvykle nevieme všetko.

- dynamická kontrola umožňuje spustiť aj nekompletný program, ktorý neošetrí všetky prípady
- statická kontrola vyžaduje ošetrenie všetkých prípadov
 - predčasný návrh finálnych dátových štruktúr
 - píšeme zbytočný dočasný kód (ktorý neskôr zahodíme), len aby sme umlčali kontrolu typov

Statická kontrola je lepšia pre prototypovanie

Statický typový systém pomáha pri prototypovaní:

- dokumentovať dátové štruktúry a kontrolovať ich správne použitie
- uistiť sa, že sa na nič nezabudlo (case)
- či sa omylom v novom kóde nespoliehame na nesprávne predpoklady

Keď ešte nie sú niektoré prípady implementované, stačí vložiť:

```
1 | _ -> error "Not implemented, yet."
```

Funkcie ako **error** :: [**Char**] -> a vracajú ľubovoľný typ.

Dynamická kontrola je lepšia pre údržbu

Môžeme kód spraviť všeobecnejší bez ovplyvnenia všetkých existujúcich volajúcich miest.

- funkcia f brala číslo a vracala číslo

```
1 (define (f x)
2   (* 2 x))
```

```
1 f x = 2 * x
```

- ale teraz môže brať aj reťazec a vracat reťazec

```
1 (define (f x)
2   (if (number? x)
3       (* 2 x)
4       (string-append x x)))
```

```
1 data T = I Int | S String
2
3 f x = case x of
4   I i -> I (2 * i)
5   S s -> S (s ++ s)
```

- Haskell – volajúci musia byť opravení, aby používali typ T
- Racket – na existujúcich volajúcich netreba nič meniť

Statická kontrola je lepšia pre údržbu

Keď sa zmení dátový typ alebo časť programu, tak statická kontrola typov vráti zoznam miest, na ktorých treba niečo opraviť.

- predchádza zaneseniu chýb
- čím viac zo špecifikácie vložíme do typov, tým viac statická kontrola typov pomôže s nájdením miest, ktoré treba opraviť, keď sa špecifikácia zmení

Príklady

- zmena návratového typu funkcie alebo jej argumentov
- pridanie nového dátového konštruktora
 - vďaka kontrole úplnosti pre pattern-matching
 - len ak nekončí všetko zachytávajúcim vzorom

Nemôžeme skompilovať program, ak sme neprešli cez všetky nájdené chyby. Znemožňuje vývoj po menších krokoch.

Koniec

Užitečné linky :-)

- [Julia Ecklar - The Eternal Flame \(text\)](#)
- [Write in C](#)
- [Write in Go](#)
- [We Code Hard](#)
- [The Python Programming Song](#)
- [Code Together](#)
- [The Software Engineer Song](#)
- [We're Gonna Build a Framework](#)

Tipy od cvičiaceho:

- [And So You Code](#)
- [MC Proxy - V těle mám rytmus](#)